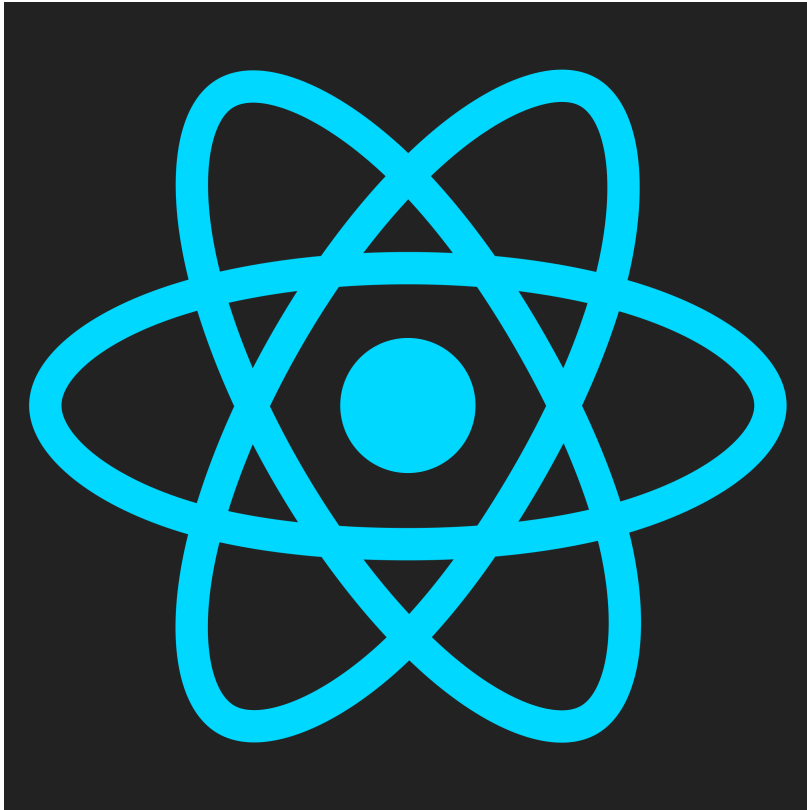


React

Thursday, June 19, 2025

11:40 AM



Props & State

Saturday, June 14, 2025 10:50 AM

Summary:

- **Props:** Passed in from parent, read-only, for configuration and data sharing.
- **State:** Managed inside the component, can change, for dynamic and interactive data. (triggers a re-render)

Props and **state** are two core concepts in React for managing data and UI updates:

Props

- **Definition:** Props (short for "properties") are read-only data passed from parent component to a child component.
- **Purpose:** They allow components to communicate and share data.
- **Usage:** A parent sets props on a child like `<Child name="John" />`, and the child accesses them as `props.name`.
- **Immutable:** A component cannot change its own props.

Example:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}  
// Usage: <Welcome name="Alice" />
```

State

- **Definition:** State is data that is managed within a component and can change over time.
- **Purpose:** It allows a component to keep track of information that may change (like user input, toggles, etc.).
- **Usage:** State is usually set up with the `useState` hook in functional components.
- **Mutable:** A component can update its own state, which triggers a re-render.

Example:

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return (  
    <button onClick={() => setCount(count + 1)}>  
      Clicked {count} times  
    </button>  
  );  
}
```

Props in React refer to properties.

They are a mechanism to pass data from a parent component to a child component.

Local State & Global State

Saturday, June 14, 2025 11:23 AM

Summary:

- **Local state** is private to a component.
- **Global state** is shared across the app and accessible by many components. (changes can be made by any component)

Local state and **global state** are two ways to manage data in a React application:

Local State

- **Definition:** Data that is managed within a single component using hooks like `useState`.
- **Scope:** Only accessible and modifiable by that component (and its children if passed as props).
- **Use cases:** Form inputs, toggles, modal visibility, temporary UI state.

Example:

```
function Counter() {  
  const [count, setCount] = useState(0); // local state  
  // Only Counter can access or change 'count'  
}
```

Global State

- **Definition:** Data that is shared across multiple components, often managed by a state management library like Redux or React Context.
- **Scope:** Accessible and modifiable by any component that subscribes to it.
- **Use cases:** User authentication, shopping cart contents, theme settings, data fetched from APIs.

Example (Redux):

```
// In Redux store  
const cart = useSelector((state) => state.cart); // global  
// Any component can access or update 'cart'
```

Hook

Thursday, June 19, 2025 11:42 AM

- Definition

A **React hook** is a special JavaScript function introduced in React 16.8 that allows you to use React features—like state, lifecycle methods, and context—inside functional components. Before hooks, only class components could use these features, but hooks make it possible in simpler, function-based components.

They are called **hooks** because they let you “hook into” (or tap into) React’s core features—such as state, lifecycle, and context—from inside your functional components. The name reflects the idea that you can “hook” additional behavior into your components without changing their structure or using class components. Hooks provide a way to reuse and share logic across components in a simple and consistent way.

Why use hooks?

- They let you reuse stateful logic across components.
- They make your components easier to read and test.
- They reduce the need for class components.

Summary

- **Hooks** let you use React features in functional components.
- They make your code more reusable and easier to maintain.
- The most common hooks are `useState`, `useEffect`, and `useContext`.

Hooks are now the standard way to write modern React components!

useState hook

Thursday, June 19, 2025 9:58 AM

How it works:

- You call `useState` inside your component and pass it the initial value for your state.
- It returns an array with two elements:
 1. The current state value.
 2. A function to update that value.

Example:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0); // 0 is the initial st

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

- `count` is the current state value.
- `setCount` is the function you use to update `count`.
- When you call `setCount`, React re-renders the component with the new value.

Summary:

`useState` lets you create and manage local state in your React functional components.

How to view in chrome dev tools

You **cannot directly view** React component state created with `useState` in the standard Chrome DevTools. However, you **can** view and inspect it using the [React Developer Tools](#) browser extension.

How to do it:

1. Install the **React Developer Tools** extension from the Chrome Web Store.
2. Open your app in Chrome.
3. Open DevTools (`F12` or `Ctrl+Shift+I`).
4. Go to the "**Components**" tab.
5. Select your component in the component tree.
6. You'll see the current state (from `useState`) and props in the right panel.

Summary:

Use the React Developer Tools extension to view and inspect component state managed by `useState` .

- Common React Hooks
 - `useState`
 - `useEffect`
 - `useContext`

use Effect hook

Thursday, June 19, 2025 11:45 AM



GitHub Copilot

The `useEffect` hook is a built-in React hook that lets you run side effects in your functional components. Side effects are operations like data fetching, subscriptions, or manually changing the DOM—things that are not directly related to rendering UI.

Basic usage:

```
import { useEffect } from "react";

useEffect(() => {
  // Code here runs after the component renders
  console.log("Component mounted or updated");
});
```

How it works:

- The function inside `useEffect` runs after every render by default.
- You can control when it runs by passing a dependency array as the second argument.

With dependencies:

```
useEffect(() => {  
  // Runs only when 'count' changes  
  console.log("Count changed!");  
}, [count]);
```

Cleanup: You can return a function from `useEffect` to clean up resources (like unsubscribing or clearing timers):

```
useEffect(() => {  
  const timer = setInterval(() => { /* ... */ }, 1000);  
  return () => clearInterval(timer); // Cleanup on unmount  
}, []);
```

Summary:

`useEffect` lets you handle side effects in React function components, similar to lifecycle methods in class components (`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`).

Dependency Array (Second Argument)

The second argument to `useEffect` is called the **dependency array**. It controls when the effect runs.

How it works:

- **No dependency array:**

```
useEffect(() => { /* ... */ });
```

The effect runs after every render.

- **Empty array (`[]`):**

```
useEffect(() => { /* ... */ }, []);
```

The effect runs only once, after the first render (like `componentDidMount`).

- **With dependencies:**

```
useEffect(() => { /* ... */ }, [count, user]);
```

The effect runs after the first render and whenever any value in the array (`count` or `user`) changes.

Why use it?

It helps you optimize performance and avoid unnecessary side effects by running the effect only when certain values change.

Example:

```
useEffect(() => {  
  // Fetch data when 'userId' changes  
  fetchUserData(userId);  
}, [userId]);
```

Here, the effect runs only when `userId` changes.

useContext hook

Thursday, June 19, 2025 11:45 AM

3. useContext

Lets you access context values (global data) in your component.

Example:

```
import { useContext } from "react";
import { ThemeContext } from "../ThemeContext";

function ThemedButton() {
  const theme = useContext(ThemeContext);
  return <button style={{ background: theme.background }}>Click
}
```

so when the page is re loaded it get erased>



GitHub Copilot

Yes, when the page is reloaded, the Redux state in memory is erased and reset to its initial state.

Unless you use a persistence solution (like saving to `localStorage` or using a library such as `redux-persist`), all Redux state will be lost on refresh and start over from the initial values defined in your reducers or slices.

So what happens when directing to another page?



GitHub Copilot

When you navigate to another page **within your React app** (using React Router or similar client-side routing), the Redux state is **not erased**. It stays in memory and is shared across all pages and components, so your app can access and update the same state as you move between pages.

Summary:

- **Page reload:** Redux state is reset.
- **Client-side navigation (routing):** Redux state persists and is available across all pages.

 GitHub Copilot

Yes, you are correct!

In React (especially with React Router), the app is a **Single Page Application (SPA)**.

When you navigate, you are not loading new HTML pages from the server—instead, you are switching between **screens** or **views** within the same page. That's why the term "**screens**" or "**components**" is often used instead of "pages" in React apps.

Summary:

- React apps are SPAs.
- Navigation switches between screens/components, not actual web pages.

So does the all the screens/components get loaded when the page is loading?

 GitHub Copilot

No, **not all screens/components are loaded at once** when the page loads.

- Only the components needed for the current route (screen) are rendered initially.
- Other screens/components are loaded and rendered **when you navigate** to them.
- With techniques like **code splitting** and **lazy loading** (`React.lazy`), you can further optimize so that even the code for other screens isn't loaded until needed.

Summary:

Only the current screen/component is loaded and rendered at first; others load as you navigate. This keeps React apps fast and efficient.

Routing

Saturday, May 17, 2025 1:18 PM

- React Router

Routing in React

Routing in reactJS is the mechanism by which we navigate through a website or web-application.

- React Router is a standard library for routing in React.
- Enables the navigation among views of various components in a React Application, allows changing the browser URL, and keeps UI in sync with the URL.
- Routing allows faster navigation between views, does not require a full page reload and it improves user experience.

Client side Routing vs Server side Routing

- Server-side routing what usually happens when you are entering a URL, the browser makes an HTTP request to the server then the server re-renders the HTML into the application
- Client side routing we first render the full react app from the server, but after that when you want to change pages, the browser uses the HTML5 **history API** to fetch the page that has already been loaded

React Router

Client-Side routing or navigation in react applications done through using React Routers.

- Installation of react router done by
 - **npm install react-router-dom**
- Basic Components of React Router
 - **BrowserRouter**: Uses the HTML5 history API to keep UI in sync with the URL.
 - **Route**: Renders UI when a location matches the route's path.
 - **NavLink**: Provides navigation links.

```
<BrowserRouter>
  <Route path="/home" component={Home} />
  <Route path="/about" component={About} />
</BrowserRouter>
```

- Install
`npm i react-router-dom`

- Index.js

```
import {
  BrowserRouter as Router,
  createRoutesFromElements,
  Route,
  RouterProvider,
  createBrowserRouter,
} from "react-router-dom";
```

- ```
const router = createBrowserRouter(
 createRoutesFromElements(
 <Route path="/" element={<App />} />
 <Route index={true} path="/" element={<HomeScreen />} />
 </Route>
)
);
```

- ```
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    { /* <App /> */ }
    <RouterProvider router={router} />
  </React.StrictMode>
);
```

- App.js

- ```
1 import { Container } from "react-bootstrap";
2 import { Outlet } from "react-router-dom";
3 import Header from "../components/Header.jsx";
4 import Footer from "../components/Footer.jsx";
5
6 const App = () => {
7 return (
8 <>
9 <Header />
10 <main className="py-3">
11 <Container>
12 <Outlet />
13 </Container>
14 </main>
15 <Footer />
16 </>
17);
18 };
```

- Install react-router-bootstrap for bootstrap related hrefs

- ```
<Navbar.Brand href="/">
  <img src={logo} alt="ProShop" />
  ProShop
</Navbar.Brand>
```

- ```
npm i react-router-bootstrap
```

# Await

Monday, May 19, 2025

6:55 PM

## What seems confusing?

js

 Copy

 Edit

```
const res = await fetch('/api/items');
```

You're told:

- `await` **pauses** the function...
- ...but JavaScript is **non-blocking**.

So you think:

If it pauses, how is it non-blocking? 🤖

✅ The key: `await` **pauses the `async` function, not the entire program.**

## 🧠 Think of it like this:

js

Copy

Edit

```
async function getData() {
 const res = await fetch('/api/data');
 console.log('Data fetched');
}

console.log('Before calling getData');
getData();
console.log('After calling getData');
```

## 🔄 What happens:

csharp

Copy

Edit

|                            |                                 |
|----------------------------|---------------------------------|
| Before calling getData     | ✅ runs immediately              |
| After calling getData      | ✅ runs immediately              |
| (Data fetching happens...) | ⌚ async task runs in background |
| Data fetched               | ✅ when fetch completes          |

# Callbacks, Promises, Async/Await

Monday, May 19, 2025

6:46 PM

## Callbacks, Promises and Async/Await

**Callbacks** - Callbacks are functions passed as arguments to other functions. They are commonly used in asynchronous operations to specify what should happen once a task is completed.

**Promises** - Promises provide a cleaner and more structured way to handle asynchronous operations. A Promise represents a value that might be available now, or in the future, or never.

**Async/Await** - The `async/await` syntax is built on top of Promises and provides a more concise and synchronous-looking way to work with asynchronous code. Asynchronous code is essential for handling tasks that involve waiting, such as I/O operations, network requests, and interactions with databases, without blocking the entire program's execution. It enables more efficient utilization of resources and better responsiveness in applications.

# Environmental Variable

Saturday, May 17, 2025 5:14 PM

- **npm i -D dotenv**
  - Set up Environment variables
  - Should end up to production at final stage
  - .env file - any global environmental variables goes here
- Why you need to setup Environment variables?

## 1. Security

You don't want to hard-code sensitive values like:

- Database credentials
- API keys (e.g., Stripe, SendGrid)
- Secret tokens (JWT secrets)
- Admin emails or passwords

Instead of this (✗ insecure):

js

Copy

Edit

```
const dbPassword = "myHardcodedPassword";
```

You use environment variables (✓ secure):

js

Copy

Edit

```
const dbPassword = process.env.DB_PASSWORD;
```

## ⚙️ 2. Configuration Flexibility

Environment variables allow different settings for:

- Development
- Testing
- Production

Example:

```
env Copy Edit

.env (for dev)
PORT=5000
NODE_ENV=development
```

On a production server, the values can change without modifying your source code.

## 📦 3. Avoid Committing Secrets

Environment variables are usually stored in a `.env` file that should be:

- Git-ignored ( `.gitignore` )
- Kept private

So, you don't accidentally leak secrets to GitHub or other VCS platforms.

## ✓ 4. Best Practices & Framework Support

Many libraries (e.g., `dotenv`) and hosting platforms (like Vercel, Heroku, Railway) use env variables by default to manage app configs safely.

### 🔧 Example: Setting Up `.env`

#### 1. Install `dotenv`:

bash

Copy

Edit

```
npm install dotenv
```

#### 2. Create `.env` file:

env

Copy

Edit

```
PORT=5000
NODE_ENV=development
```

#### 3. Load it in your app:

js

Copy

Edit

```
import dotenv from "dotenv";
dotenv.config();

const port = process.env.PORT || 5000;
```



# Fetching Data

Saturday, May 17, 2025 5:17 PM

- You can use fetch api or axios.
  - Axios is full featured and easier.
- npm i axios
- Add a proxy (it needs in development, doesn't get end up in production)

## Why You Need a Proxy

### 1. To Avoid CORS Errors

When your frontend (e.g. on `http://localhost:3000`) tries to fetch data from a backend (e.g. `http://localhost:5000`), the browser blocks it due to **CORS (Cross-Origin Resource Sharing)**.

A **proxy** acts as a bridge so the browser thinks everything is coming from the same origin.

### 2. Simplifies API Calls

Without proxy:

```
js axios.get("http://localhost:5000/api/products");
```

With proxy:

```
js axios.get("/api/products");
```

The proxy automatically forwards `/api/*` calls to the backend.

## Example

If your Express backend has:

js

 Copy

 Edit

```
app.get('/api/products', (req, res) => {
 res.json([...]);
});
```

Then from React:

js

 Copy

 Edit

```
axios.get('/api/products') // Automatically goes to http://localhost:5000/api/products
```

# Key Prop

Saturday, May 31, 2025 2:35 PM

```
<option key={x + 1} value={x + 1}>
 {x + 1}
</option>
```

- The `key` prop in React is used to uniquely identify each element in a list when rendering multiple elements (like inside `.map()`).  
In your code:
- This helps React efficiently update and re-render only the changed items in the list, improving performance and avoiding rendering bugs.

# Undefined

Wednesday, July 16, 2025 7:34 PM

```
undefined
undefined
undefined
undefined
Content Page Weava init: 2.0.44 prod
▶ (6) [(-), (-), (-), (-), (-), (-)] ❶
▶ 0: {_id: '682b728497d24c7378f38d94', user: '682b728497d24c7378f38d8e', name: 'Cannon EOS 80D DSLR Camera', image: '/images/camera.jpg', brand: 'Cannon', _}
▶ 1: {_id: '682b728497d24c7378f38d93', user: '682b728497d24c7378f38d8e', name: 'iPhone 11 Pro 256GB Memory', image: '/images/phone.jpg', brand: 'Apple', _}
▶ 2: {_id: '682b728497d24c7378f38d92', user: '682b728497d24c7378f38d8e', name: 'Airpods Wireless Bluetooth Headphones', image: '/images/airpods.jpg', brand: 'Apple', _}
▶ 3: {_id: '682b728497d24c7378f38d95', user: '682b728497d24c7378f38d8e', name: 'Sony Playstation 4 Pro White Version', image: '/images/playstation.jpg', brand: 'Sony', _}
▶ 4: {_id: '682b728497d24c7378f38d97', user: '682b728497d24c7378f38d8e', name: 'Amazon Echo Dot 3rd Generation', image: '/images/alexa.jpg', brand: 'Amazon', _}
```

- Get undefined because the page still loading.
  - That's why we are checking isLoading

# Redux

Wednesday, May 21, 2025 9:58 PM

- Redux is mainly used to **manage global state** in large or complex React applications. It provides a single source of truth for your app's data, making it easier to:
  - **Share state** between many components
  - **Predictably update state** using actions and reducers
  - **Debug and track changes** with tools like Redux DevTools
  - **Organize state logic** in a scalable way
- **Redux state is always global.**
- **Redux is mainly used to manage global state in large or complex React applications.**
- Can I have local state in Redux?
  - No, you cannot have true **local state** in Redux.  
Redux is designed for **global state**—state that is shared across your entire application.
  - **Local state** (state that is only used within a single component) should be managed with React's `useState` or `useReducer` hooks, not Redux.  
Redux state is always global and accessible from any connected component.

## State

- Component State
  - Use State
- App/global State

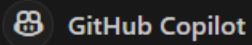
State management refers to the way that we manage data that our application needs to keep track of.

## Files created

- Store.js
- Constant.js
- Index.js (modified)

## Slices

- It's a way to organize state
- Collections of reducers and actions that are related to each other.
- We can have multiple slices in our application and each slice can have own state.



**Redux state** is a single JavaScript object (the "store") that holds all the global state for your application. Here's how it works and how it originates as your app runs:

### 1. Store Creation (Origin)

When your app starts, you create the Redux store using a function like `configureStore` or `createStore`.

You provide it with your reducers and the **initial state** (either explicitly or via each reducer's default state).

```
const store = configureStore({ reducer: rootReducer
```

- The **initial state** is set by each reducer's default value or the `initialState` you define in your slices.

## 2. State Structure

The Redux state is a single object, but it can have many "slices" (sections), each managed by a different reducer.

For example:

```
{
 cart: { cartItems: [], total: 0 },
 user: { name: '', loggedIn: false },
 products: { items: [], loading: false }
}
```

1 vulnerability >

## 3. How State Changes

- Components **dispatch actions** (like `addToCart` ).
- Redux sends the current state and the action to the appropriate reducer.
- The reducer returns a **new state** based on the action.
- The Redux store updates, and any subscribed components re-render with the new state.

## 4. Lifecycle

- The Redux state is created when the app (and store) starts.
- It lives in memory as long as the app is running.
- When you refresh or close the app, the Redux state is lost (unless you persist it to localStorage or another storage).

### Summary:

Redux state originates from the initial state provided by your reducers when the store is created. It is a single, global object that updates in response to dispatched actions, ensuring a predictable and centralized way to manage your app's data as the server (or client app) runs.

# Redux Storage & Local Storage

Saturday, June 14, 2025 3:33 PM

- is Local storage and Redux store same?

No, **local storage** and the **Redux store** are not the same.

- **Redux store** is an in-memory JavaScript object that holds your app's state while the app is running. It is fast and temporary—when you refresh the page, the Redux store is reset.
- **Local storage** is a browser feature that saves data as strings on the user's device. It persists even after a page reload or browser restart.

## Summary:

- Redux store = temporary, in-memory state
- Local storage = persistent, browser-based storage

# Context API of React

Saturday, June 14, 2025 2:46 PM

You can use context Api from react instead Redux for global state management. But it has its own pros and cons.

## Pros of Context API

- **Built-in:** No extra dependencies; it's part of React.
- **Simple:** Great for small to medium apps or for sharing simple data (like theme, user, or locale).
- **Less Boilerplate:** Fewer files and less setup than Redux.

## Cons of Context API

- **Performance:** Frequent or large updates can cause unnecessary re-renders of all consuming components, which can slow down your app.
- **Not for Complex State:** Lacks built-in tools for advanced state logic, middleware, or async actions (like Redux's middleware or RTK Query).
- **Debugging:** No built-in devtools for tracking state changes and actions.
- **Scalability:** Can become hard to manage as your app grows and state logic becomes more complex.

# Slice

Wednesday, June 11, 2025 6:22 PM

- A **slice** is a section of your Redux state and all the logic (reducers and actions) related to it.
- A slice is a section of your Redux state
- Each Slice contain
  - **Initial state**
  - Reducer Function - **handle actions & update the state**
  - **Dispatch actions**

What is a slice?

- A **slice** represents a single "slice" (section) of the Redux state, such as `cart`, `user`, or `products`.
- Each slice contains:
  - The initial state for that part of the app
  - Reducer functions to handle actions and update the state
  - Action creators for dispatching actions

Example

A `cartSlice` manages cart state, a `userSlice` manages user state, etc.

In your code

You are using `createApi` from Redux Toolkit Query to create an

- **apiSlice:**

```
export const apiSlice = createApi({
 baseQuery,
 tagTypes: ["Product", "Order", "User"],
 endpoints: (builder) => ({}),
});
```

This `apiSlice` manages all API-related state and logic for your app.

**Summary:**

A **slice** in Redux is a focused module for a specific part of your state, containing its state, reducers, and actions, making your Redux code more organized and maintainable.

# Traditional Redux Slice vs RTK Query API Slice

Sunday, June 15, 2025 10:18 AM

## 1. [cartSlice.js](#) — Traditional Redux Slice

- **Purpose:**  
Manages the cart's state (cart items, add/remove logic, syncing with localStorage) using Redux Toolkit's [createSlice](#).
- **How it works:**
  - Uses [createSlice](#) to define a **slice of state** called "cart".
  - Contains **reducers** ([addToCart](#), [removeFromCart](#)) that handle synchronous updates to the cart state.
  - Exports **action creators** and the **reducer** for use in the Redux store.
- **Data flow:**
  - All logic is local and synchronous.
  - No built-in support for async requests or API calls.

## 2. [productApiSlice.js](#) — RTK Query API Slice

- **Purpose:**  
Manages **API data fetching and caching** for products using Redux Toolkit Query (RTK Query).
- **How it works:**
  - Uses [apiSlice.injectEndpoints](#) to define endpoints for fetching products and product details.
  - Handles **asynchronous requests** (e.g., fetching from a backend API).
  - Automatically manages caching, loading, and error states.
  - Exports **auto-generated hooks** (useGetProductsQuery, useGetProductDetailsQuery) for use in React components.
- **Data flow:**
  - Logic is centered around **async API calls**.
  - RTK Query handles caching, invalidation, and background updates for you.

## Why the Difference?

- **Traditional slices** ([createSlice](#)) are best for managing local/global app state that is updated synchronously (like a cart, UI toggles, or form data).
- **API slices** (RTK Query) are specialized for managing **remote data**: fetching, caching, and updating data from a server, with minimal boilerplate.

## Summary Table

File	Type	Handles	Example Usage
<a href="#">cartSlice.js</a>	Traditional Slice	Local/global state	Cart items, add/remove logic
<a href="#">productApiSlice.js</a>	RTK Query API Slice	Remote API data	Fetching products

**In short:**

Both are called "slices" because they manage a "slice" of your Redux state, but one is for local logic (cart), and the other is for async API data (products). This separation keeps your code organized and leverages the strengths of Redux Toolkit and RTK Query.

productApiSlice.js — RTK Query API Slice, does this maintain a state of application since slice idea is for managing states.?

Yes, productApiSlice.js (an RTK Query API slice) **does maintain a slice of your application's state**, but in a different way than a traditional Redux slice.

- **RTK Query API slices** automatically manage state related to data fetching, such as:
  - The fetched data (e.g., product lists, product details)
  - Loading status (isLoading)
  - Error status (error)
  - Caching and invalidation

This state is stored in your Redux store under a key (like api or whatever you set as reducerPath).

You can access and use this state in your components via the auto-generated hooks (e.g., useGetProductsQuery).

**Summary:**

Yes, an RTK Query API slice manages a part of your Redux state, but it focuses on API data, loading, and error status, rather than local app logic.

# Deciding a global state

Saturday, June 14, 2025 2:51 PM

- **Does more than one unrelated component need to access or update this state?**  
If yes, it's a good candidate for global state.
- **Is the state needed throughout many parts of the app (e.g., user authentication, theme, cart)?**  
If yes, global state is appropriate.
- **Is the state only used within a single component or its direct children?**  
If yes, keep it as local state.
- **Does the state need to persist across routes or after a page reload?**  
If yes, consider global state (and possibly localStorage or Redux Persist).

## Summary:

- Use **local state** for UI details and component-specific data.
- Use **global state** for data shared across many components or that affects the whole app.

## Why Cart Item could be a global state

- The **cart** is a good example of global state because it needs to be accessed and updated by multiple, often unrelated, components throughout an e-commerce application. For example, the product page, cart page, header (showing cart count), and checkout page all need to read from or update the cart. By storing the cart as global state, you ensure that all parts of the app always see the current cart contents, and any updates are reflected everywhere instantly. This makes the user experience consistent and the state management much simpler as your app grows.

# Redux: useSelector

Saturday, May 31, 2025 3:51 PM

- // using useSelector to access the cart state from the Redux store
- // useSelector is a hook that allows you to extract data from the Redux store state
- // it takes a function as an argument that receives the entire state and returns the part of the state you want
- // in this case, we are extracting cartItems from the cart slice of the Redux store
- // cartItems is an array of items in the cart, which is managed by the cartSliceReducer
- // cartItems is coming from the store.js file where we have defined the cartSliceReducer

```
const { cartItems } = useSelector((state) => state.cart); // cart is
coming from store.js carSliceReducer
console.log(cartItems);
```

# Refetch function()

Saturday, July 12, 2025 10:15 AM

```
const {
 data: order,
 refetch, // Refetch the order details wh
 isLoading,
 error,
} = useGetOrderDetailsQuery(orderId);
```

```
const deliverOrderHandler = async () => {
 try {
 await deliverOrder(orderId);
 refetch(); // Refetch the order details after delivery
 toast.success("Order delivered");
 } catch (err) {
 toast.error(err?.data?.message || err.message);
 }
};
```

# Fetching Data

Sunday, July 20, 2025 3:34 PM

- Fetch()
- RTK QUERY
  - InvalidatesTags[""]
  - providedTags[""]

```
updateProduct: builder.mutation({
 query: (data) => ({
 url: `${PRODUCTS_URL}/${data.productId}`,
 method: "PUT",
 body: data,
 }),
 invalidatesTags: ["Products"],
}),
});
```

```
export const productsApiSlice = apiSlice.injectEndpoints({
 // two endpoints are added to the API using injectEndpoints m
 endpoints: (builder) => ({
 // getProducts is query endpoint.
 getProducts: builder.query({
 query: () => ({
 url: PRODUCTS_URL, // Doesn't need to perform fetch req
 }),
 keepUnusedDataFor: 5, // 5 Seconds // This is how long RT
 providesTags: ["Products"],
 }),
 }),
});
```

use of providesTags: ["Products"], ?

JS productsApiSlice.js:14

> Used 1 reference

The `providesTags: ["Products"]` line in your RTK Query endpoint tells Redux Toolkit Query that the data returned by the `getProducts` query should be associated with the `"Products"` tag in the cache.

### Why is this useful?

When you later perform a mutation (like creating or updating a product) and use `invalidatesTags: ["Products"]`, RTK Query knows to automatically refetch any queries that provide the `"Products"` tag. This keeps your product list up-to-date without manual intervention.

### Summary:

- `providesTags: ["Products"]` marks the cached data from this query with the `"Products"` tag.
- Mutations that `invalidate` this tag will trigger a refetch of the product list, ensuring fresh data in your UI.

should we still need to use fetch() though?

JS productsApiSlice.js:1-21

> Used 1 reference

No, you **do not need to use** `fetch()` directly when using RTK Query in your Redux slices.

RTK Query automatically handles all the network requests for you.

You just define your endpoints (like `getProducts`, `getProductDetails`, etc.), and then use the auto-generated hooks (like `useGetProductsQuery`) in your React components.

#### Example usage in a component:

```
const { data: products, isLoading, error } = useGetProductsQuery();
```

This hook will automatically fetch data from your API—no need to call `fetch()` or `axios` manually.

#### Summary:

- RTK Query replaces the need for manual `fetch()` calls.
- Just use the hooks generated by your API slice.

if i use `fetch()` should i use `providesTags` and `invalidateTags`?

JS productsApiSlice.js:1-21

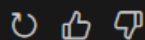
> Used 1 reference

If you use `fetch()` directly (not RTK Query), you **do not** use `providesTags` or `invalidatesTags`.

These features are specific to RTK Query and are used for automatic cache management and refetching.

#### Summary:

- `providesTags` and `invalidatesTags` are only for RTK Query endpoints.
- If you use `fetch()`, you must manually manage cache and refetching in your app.
- For automatic cache updates and less boilerplate, use RTK Query instead of raw `fetch()`.



# JavaScript

Thursday, June 19, 2025

11:47 AM

# Arrow Functions

Monday, May 19, 2025 6:16 PM

## JavaScript Arrow Function

### Before Arrow

```
hello = function() {
 return "Hello World!";
}
```

### With Arrow

```
hello = () => {
 return "Hello World!";
}
```

### Arrow Functions Return Value by Default

```
hello = () => "Hello World!";
```

### Arrow Functions With Parameters

```
hello = (val) => "Hello " + val;
```

**If you have only one parameter, you can skip the parentheses as well:**

```
hello = val => "Hello " + val;
```

# Return

Monday, July 28, 2025

5:25 PM

does arrow function automatically return a value

ProductScreen.jsx:60-83

> Used 1 reference

Arrow functions **do not always automatically return a value**—it depends on their syntax:

## 1. Implicit Return (No Curly Braces)

If you write an arrow function **without curly braces**, the value of the expression is returned automatically:

```
const add = (a, b) => a + b; // Implicit return
// Equivalent to: return a + b;
```

## 2. Explicit Return (With Curly Braces)

If you use **curly braces** `{}`, you must use the `return` keyword to return a value:

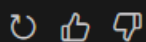
```
const add = (a, b) => {
 return a + b; // Explicit return
};
```

If you omit `return` inside curly braces, the function returns `undefined`:

```
const add = (a, b) => {
 a + b; // No return, returns undefined
};
```

## Summary:

- **No curly braces:** returns the expression automatically.
- **With curly braces:** you must use `return` to return a value.



## Compare With:

### 1. Without curly braces — implicit return:

javascript

 Copy  Edit

```
const myFunc = (e) => setComment(e.target.value);
```

- If you **don't use curly braces**, the expression is **returned implicitly**.
- This is shorter and common when you only call one function.

### 2. With curly braces — no return unless specified:

javascript

 Copy  Edit

```
const myFunc = (e) => {
 setComment(e.target.value);
 // more lines could go here
};
```

- Now you have a block body.
- You can add **more logic** inside, and it won't automatically return anything.

# Callback function

Saturday, May 17, 2025 1:15 PM

JS index.js

```
1 // callback = a function that is passed as an argument
2 // to another function.
3
4 // used to handle asynchronous operations:
5 // 1. Reading a file
6 // 2. Network requests
7 // 3. Interacting with databases
8
9 // "Hey, when you're done, call this next."
```

```
hello(goodbye);

function hello(callback){
 console.log("Hello!");
 callback();
}

function goodbye(){
 console.log("Goodbye!");
}
```

```
function greet(name) {
 console.log("Hello, " + name);
}

function processUserInput(callback) {
 const name = "Alice";
 callback(name); // greet is used as a callback here
}

processUserInput(greet);
```

Copy Edit

### ✓ Answer:

The function `greet` is the **callback function**.

### ✓ Why?

Because it's **passed as an argument** to `processUserInput` and **invoked inside** `processUserInput`.

So in this line:

```
processUserInput(greet);
```

Copy Edit

- `greet` is passed as a **callback** to `processUserInput`.
- Inside `processUserInput`, the parameter `callback` is used to call `greet(name)`.

💡 **Definition Reminder:** A callback function is **any function passed to another function as an argument**, which is then **invoked inside that other function**.



Let me know if you want an example where the callback is written **inline** (e.g., anonymous function or arrow

- Immediate call & not

### Summary:

- `onClick={addToCartHandler}` → runs when clicked (**correct**)
- `onClick={addToCartHandler()}` → runs immediately on render (**incorrect** for event handlers)

## Explanation:

js

Copy

Edit

```
app.listen(port, () => console.log(`Server running on port ${port}`));
```

- `app.listen(port, callback)` is an **Express method** that starts the server.
- The second argument, `() => console.log(...)`, is a **callback function**.
  - It runs **after the server starts successfully**.

## Callback Function Definition

A **callback function** is a function passed as an **argument** to another function, to be **executed later**.

So in this case:

- `app.listen(...)` runs
- Once the server is listening, it **calls the provided function** → `() => console.log(...)`

A function that is passed as an argument to another function, and is called (or "called back") inside that function at a certain point.

- Ex:

```
const product = products.find(function(p) {
 return p._id === productId;
});
```

```
array.find(callbackFunction);
```

- JavaScript **automatically passes each item in the array** to your callback function as an argument — that's what `p` (or whatever name you choose) is.

- Hence

```
const product = products.find((p) => p._id === productId);
```

# How addToCartHandler is a callback function

Monday, July 28, 2025

3:22 PM

how you are saying addToCartHandler is a callback function

ProductScreen.jsx:66

> Used 1 reference

`addToCartHandler`

is a **callback function** because it is **passed as a value to the** `onClick` **prop** of the `<Button>` component:

```
<Button
 className="btn-block"
 type="button"
 disabled={product.countInStock === 0}
 onClick={addToCartHandler}
>
 Add To Cart
</Button>
```

Here's why:

- The `onClick` prop expects a function to be called **when the button is clicked**.
- You are passing the function reference `addToCartHandler` (not calling it immediately).
- React will call (invoke) `addToCartHandler` **later**, when the user clicks the button.

**This is the definition of a callback function:**

A function passed as an argument (or prop) to another function/component, to be called at a later time or in response to an event.

**Summary:**

`addToCartHandler` is a callback function because it is passed to `onClick` and will be executed by React when the button is clicked.

Why not calling immediately



why not calling it immediately?

ProductScreen.jsx:66

> Used 1 reference

You do **not** call `addToCartHandler` immediately (with `addToCartHandler()`) because you want it to run **only when the user clicks the button**, not when the component renders.

When you write:

```
<Button onClick={addToCartHandler}>Add To Cart</Button>
```

You are passing the **function reference** to React. React will call (invoke) `addToCartHandler` for you when the click event happens.

If you wrote:

```
<Button onClick={addToCartHandler()}>Add To Cart</Button>
```

This would **call the function immediately** during rendering, not on click, and the result (likely `undefined`) would be assigned to `onClick`, so nothing would happen when the button is clicked.

#### Summary:

- `onClick={addToCartHandler}` → runs when clicked (**correct**)
- `onClick={addToCartHandler()}` → runs immediately on render (**incorrect** for event handlers)

# Reduce array function

Sunday, June 1, 2025

10:40 AM

- `<h2>Subtotal ({cartItems.reduce((acc, item) => acc + item.qty, 0)})</h2>`

```
cartItems.reduce((acc, item) => acc + item.qty, 0)
```

## 1. What is `reduce()` ?

- `reduce()` is a JavaScript array method.
- It processes each element in the array and accumulates a single result.
- It takes a callback function and an initial value.

## 2. Callback Function: `(acc, item) => acc + item.qty`

- `acc` stands for **accumulator**. It stores the running total as the array is processed.
- `item` is the **current element** (in this case, a cart item object) being processed in the array.
- `acc + item.qty` means: add the current item's `qty` (quantity) to the accumulator.

## 3. Initial Value: `0`

- The second argument to `reduce()` is the initial value of the accumulator.
- Here, it starts at `0`.

#### 4. How it works step by step:

Suppose `cartItems` looks like this:

```
[
 { name: "Apple", qty: 2 },
 { name: "Orange", qty: 3 }
]
```

- **First iteration:**

`acc = 0` , `item = { name: "Apple", qty: 2 }`

`acc + item.qty` → `0 + 2 = 2`

- **Second iteration:**

`acc = 2` , `item = { name: "Orange", qty: 3 }`

`acc + item.qty` → `2 + 3 = 5`

**Final result:** `5` (total quantity in the cart)

#### Summary:

- `reduce()` loops through all cart items.
- `acc` keeps the running total.
- `item.qty` is added to `acc` for each item.
- `0` is the starting value.
- The result is the total quantity of all items in the cart.

# ering in React

Saturday, May 31, 2025

1:53 PM

```
<Form.Control
 as="select"
 value={qty}
 onChange={(e) => setQty(Number(e.target.value))}
>
 {[...Array(product.countInStock).keys()].map((x) => (
 <option key={x + 1} value={x + 1}>
 {x + 1}
 </option>
))}
</Form.Control>
```

- [Array\(product.countInStock\)](#) creates an empty array with a length equal to [product.countInStock](#) with empty slots.
- `[...Array(...)]` spreads that array into a new array (still empty, but with values set to undefined).
- `.keys()` returns an iterator of the array's indexes (from 0 to [countInStock - 1](#)).
- Wrapping with `[...]` turns that iterator into an array of numbers: `[0, 1, 2, ..., countInStock - 1]`.
- `.map((x) => ...)` lets you render an `<option>` for each quantity, starting from 1 up to [countInStock](#).

This is commonly used to generate a range of numbers for mapping, such as rendering `<option>` elements for a select dropdown. However, you should map over this array to actually render the options:

----Another Explanation -----

```
[...Array(product.countInStock).keys()]
```

- `Array(product.countInStock)` creates an empty array with a length equal to the number of items in stock.
- `[...Array(...)]` uses the spread operator to turn that array into a real, iterable array.
- `.keys()` returns an iterator of the array's indices (from 0 up to `countInStock - 1`).
- The result is an array like `[0, 1, 2, ..., countInStock - 1]`.

```
.map((x) => (
 <option key={x + 1} value={x + 1}>
 {x + 1}
 </option>
))
```

- `.map((x) => ...)` loops over each index in the array.
- For each `x`, it creates an `<option>` element:
  - `key={x + 1}` gives each option a unique key (React requires this for lists).
  - `value={x + 1}` sets the value of the option to a number from 1 up to `countInStock`.
  - `{x + 1}` inside the option displays the number to the user.

----- Detailed Explanation -----

**1. `Array(product.countInStock)` creates an empty array with a length equal to the number of items in stock.**

- If `product.countInStock` is 5, this creates an array like `[empty × 5]`.
- The array has 5 slots, but each slot is empty (not even `undefined`), so you can't directly map over it.

**2. `[...Array(...)]` uses the spread operator to turn that array into a real, iterable array.**

- The spread operator `(...)` takes each slot in the array and "spreads" it into a new array.
- This makes the array iterable, so you can use methods like `.map()` or `.keys()` on it.

**3. `.keys()` returns an iterator of the array's indices (from 0 up to `countInStock - 1`).**

- The `.keys()` method gives you an iterator that produces the index of each element in the array.
- For an array of length 5, `.keys()` will produce: `0, 1, 2, 3, 4`.

**4. The result is an array like `[0, 1, 2, ..., countInStock - 1]`.**

- When you spread the result of `.keys()` into an array, you get a list of numbers starting at 0 and ending at one less than `countInStock`.
- For example, if `countInStock` is 5, you get `[0, 1, 2, 3, 4]`.

# Pagination

Wednesday, July 30, 2025 9:06 PM

```
const Paginate = ({ pages, page, isAdmin = false }) => { 'pages' is missing in props validation
 return (
 <Pagination>
 { /* Generate number of pagination - Refer note */ }
 [...Array(pages).keys()].map((x) => (
 <Pagination.Item
 as={Link}
 key={x + 1}
 to={!isAdmin ? `/page/${x + 1}` : `/admin/productlist/${x + 1}`}
 active={x + 1 === page}
 >
 {x + 1}
 </Pagination.Item>
)))
 </Pagination>
)
);
};

export default Paginate;
```

This code snippet is a common JavaScript and React pattern for generating a sequence of elements based on a numeric value, in this case, pages.

@workspace /explain Write an explanation for the active selection as paragraphs of text.

```
{[...Array(pages).keys()].map((x) => (
```

Paginate.jsx:8

## Workspace

> Used 1 reference

This code snippet is a common JavaScript and React pattern for generating a sequence of elements based on a numeric value, in this case, `pages`.

Here's how it works:

- `Array(pages)` creates a new array with a length equal to the value of `pages`. However, this array is initially filled with empty slots.
- `[...Array(pages).keys()]` uses the `.keys()` method, which returns an iterator containing the keys (indexes) for every element in the array, from `0` up to `pages - 1`. The spread operator `(...)` then converts this iterator into a real array of numbers, like `[0, 1, 2, ..., pages - 1]`.
- `.map((x) => (...))` iterates over each number in this array, allowing you to render something for each page index. In a pagination component, this is typically used to render a button or link for each page.

In summary, this pattern is used to dynamically render a list of elements (such as pagination links) based on the number of pages, ensuring that the UI adapts automatically if the number of pages changes.

# Overview: connection between files

Wednesday, July 30, 2025 10:57 PM

## 1. Index.js

```
JS index.js 1, M X HomeScreen.jsx 3, M JS productsApiSlice.js U JS productController.js M Paginate.jsx 4, U
frontend > src > JS index.js > router
42 // This nested structure allows the App component to provide shared layout or logic for all these pages.
43
44 const router = createBrowserRouter(
45 createRoutesFromElements(
46 <Route path="/" element={<App />} />
47 <Route index={true} path="/" element={<HomeScreen />} />
48 <Route path="/page/:pageNumber" element={<HomeScreen />} />
49 <Route path="/search/:keyword" element={<HomeScreen />} />
50 <Route
51 path="/search/:keyword/page/:pageNumber"
52 element={<HomeScreen />}
53 />
54 <Route path="/product/:id" element={<ProductScreen />} />
55 <Route path="/cart" element={<CartScreen />} />
56 <Route path="/login" element={<LoginScreen />} />
57 <Route path="/register" element={<RegisterScreen />} />
58
59 <Route path="" element={<PrivateRoute />} />
60 <Route path="/shipping" element={<ShippingScreen />} />
61 <Route path="/payment" element={<PaymentScreen />} />
62 <Route path="/placeorder" element={<PlaceOrderScreen />} />
63 <Route path="/order/:id" element={<OrderScreen />} />
64 <Route path="/profile" element={<ProfileScreen />} />
65 </Route>
66)

```

## 2. Homescreen.jsx

```

JS index.js 1, M X HomeScreen.jsx 3, M X JS productsApiSlice.js U JS productController.js M Paginate.jsx 4, U
frontend > src > screens > HomeScreen.jsx > HomeScreen
9 | import Paginate from "../components/Paginate";
10
11 const HomeScreen = () => {
12 /* const [products, setProducts] = useState([]);
13
14 useEffect(() => {
15 const fetchProducts = async () => {
16 const { data } = await axios.get("/api/products");
17 setProducts(data);
18 };
19
20 fetchProducts();
21 }, []); */
22
23 const { keyword, pageNumber } = useParams();
24
25 const { data, isLoading, error } = useGetProductsQuery({
26 keyword,
27 pageNumber,
28 }); //RTK
29
30 return (
31 // format - isLoading ? () : error ? () : ()
32 <>
33 {isLoading ? (

```

### 3. ProductApiSlice.js

```

JS index.js 1, M X HomeScreen.jsx 3, M JS productsApiSlice.js U JS productController.js M Paginate.jsx 4, U
frontend > src > slices > JS productsApiSlice.js > productsApiSlice > endpoints > getProducts > query > params
1 | import { PRODUCTS_URL, UPLOAD_URL } from "../constants";
2 | import { apiSlice } from "../apiSlice"; // apiSlice - use when having endpoints that are d
3 | // This is a RTK Query API Slice: specialized for managing remote data: fetching, catchin
4 |
5 | export const productsApiSlice = apiSlice.injectEndpoints({
6 | // two endpoints are added to the API using injectEndpoints method of apiSlice.
7 | endpoints: (builder) => ({
8 | // getProducts is query endpoint.
9 | getProducts: builder.query({
10 | query: ({ keyword, pageNumber }) => ({
11 | url: PRODUCTS_URL,
12 | params: {
13 | keyword,
14 | pageNumber,
15 | }, // Doesn't need to perform fetch request or axios request to, instead done thr
16 | }),
17 | keepUnusedDataFor: 5, // 5 Seconds // This is how long RTK Query will keep your dat
18 | providesTags: ["Products"],
19 | // provideTags: ["Products"] associates the query's cached data with the "Products

```

### 4. productController.js

```

JS index.js 1, M HomeScreen.jsx 3, M X JS productsApiSlice.js U JS productController.js M Paginate.jsx 4, U
frontend > src > screens > HomeScreen.jsx > HomeScreen
11 const HomeScreen = () => {
12
13 <Row>
14 {data.products.map((product) => (
15 <Col key={product._id} sm={12} md={6} lg={4} xl={3}>
16 <Product product={product} />
17 </Col>
18)]}
19 </Row>
20 <Paginate
21 pages={data.pages}
22 page={data.page}
23 keyword={keyword ? keyword : ""}
24 />
25 </>
26)};
27 };
28
29 export default HomeScreen;
30

```

## 5. Paginate.jsx

```

JS index.js 1, M HomeScreen.jsx 3, M JS productsApiSlice.js U JS productController.js M Paginate.jsx 4, U X
frontend > src > components > Paginate.jsx > Paginate > map() callback
3
4 const Paginate = ({ pages, page, isAdmin = false, keyword = "" }) => { 'pages' is missing in props validation
5 return (
6 pages > 1 && (
7 <Pagination>
8 /* Generate number of paginates - Refer note */
9 [...Array(pages).keys()].map((x) => (
10 <Pagination.Item
11 as={Link}
12 key={x + 1}
13 to={
14 !isAdmin
15 ? keyword
16 ? `/search/${keyword}/page/${x + 1}`
17 : `/page/${x + 1}`
18 : `/admin/productlist/${x + 1}`
19 }
20 active={x + 1 === page}
21 >
22 {x + 1}
23 </Pagination.Item>
24))
25 </Pagination>
26)
27)

```

# Event Object in Event Listener

Saturday, May 31, 2025 2:38 PM

```
onChange={(e) => setQty(Number(e.target.value))}
```

- (e) event object passed to the onChange handler of <Form.Control> component
- In React, when an input changes, the event handler receives an event object (e) that contains information about the change, such as the new value.

# Spread Across Operator

Monday, June 23, 2025

9:59 PM

```
1 import { apiSlice } from "../apiSlice";
2 import { ORDERS_URL } from "../constants";
3
4 export const orderApiSlice = apiSlice.injectEndpoints({
5 endpoints: (builder) => ({
6 createOrder: builder.mutation({
7 query: (order) => ({
8 url: ORDERS_URL,
9 method: "POST",
10 // The spread operator flattens the order object into the request body
11 // It ensures that all key-value pairs from the order object are included
12 body: { ...order },
13 }),
14 }),
15 });
16
17 export const { useCreateOrderMutation } = orderApiSlice;
```

The spread operator (`...order`) is used here to create a **shallow copy** of the `order` object and include all its properties in the request body:

```
body: { ...order },
```

## Why use it?

- It ensures that all key-value pairs from the `order` object are sent as individual fields in the request body, not as a nested object.
- This is useful if you want the backend to receive a flat object with all order details, rather than `{ order: { ... } }`.

### Example:

If `order` is:

```
{
 items: [/*...*/],
 shippingAddress: { /*...*/ },
 total: 100
}
```

Then `body: { ...order }` results in:

```
{
 items: [/*...*/],
 shippingAddress: { /*...*/ },
 total: 100
}
```

instead of:

```
{
 order: {
 items: [/*...*/],
 shippingAddress: { /*...*/ },
 total: 100
 }
}
```

### Summary:

- The spread operator flattens the `order` object into the request body, matching what your backend expects.

# Curly braces & Parentheses

Monday, July 28, 2025

12:11 PM

```
const addToCartHandler = () => {
 dispatch(addToCart({ ...product, qty })
 // The syntax { ...product, qty } uses
 navigate("/cart");
};
```

why this is not a paranthese but curly brace?

ProductScreen.jsx:64-68

> Used 1 reference

Great question!

In your code:

```
{
 dispatch(addToCart({ ...product, qty }));
 navigate("/cart");
}
```

This is **not** a callback function or an object.

It is a **block of code** (a code block), not a function argument or a value.

Why curly braces `{ }` and not parentheses `( )`?

- **Curly braces** `{ }` are used in JavaScript to define a block of statements, such as the body of a function, an if statement, or a loop.
- **Parentheses** `( )` are used for function calls or to group expressions.

**Example:**

```
const myFunction = () => {
 // This is a function body, so we use curly br
 console.log("Hello");
};
```

If you wrote:

```
(dispatch(addToCart({ ...product, qty })), navig
```

This would be a grouped expression, not a block of code.

**Summary:**

You use curly braces `{ }` to wrap multiple statements in a function or handler.

Parentheses `()` are for function calls or grouping expressions.

In your code, curly braces are correct because you are writing multiple statements inside a function.

↩ ⏪ ⏩ ⏹

# Back End

Sunday, June 15, 2025

12:24 PM

# Home Screen Loading

Sunday, June 15, 2025 12:24 PM

```
const store = configureStore({
 reducer: {
 [apiSlice.reducerPath]: apiSlice.reducer,
 cart: cartSlice.reducer,
 },
 middleware: (getDefaultMiddleware) =>
 getDefaultMiddleware().concat(apiSlice.middleware),
 devTools: true, // process.env.NODE_ENV !== "production"
});

const root = ReactDOM.createRoot(document.getElementById("root"));
// This method is responsible for rendering your React component tree into
// This is the entry point for rendering your entire React application and
root.render(
 <React.StrictMode>
 <Provider store={store}>
 <RouterProvider router={router} />
 </Provider>
 </React.StrictMode>
);

reportWebVitals();

const router = createBrowserRouter(
 createRoutesFromElements(
 <Route path="/" element={App} />
 <Route index={true} path="/" element={HomeScreen} />
 <Route path="/product/:id" element={ProductScreen} />
 <Route path="/cart" element={CartScreen} />
)
);

const HomeScreen.jsx
```

```
const app = express();

// Body parser middleware
app.use(express.json()); // parses application/json
app.use(express.urlencoded({ extended: true }));

// Cookie parser middleware
app.use(cookieParser()); // parses cookies attached to requests

app.get('/', (req, res) => {
 res.send('API is running...');
});

app.use("/api/products", productRoutes); // It takes care of routing
app.use("/api/users", userRoutes);

import express from "express";
const router = express.Router();

import {
 getProducts,
 getProductById,
} from "../controllers/productController.js";

router.route("/").get(getProducts);
router.route("/:id").get(getProductById);

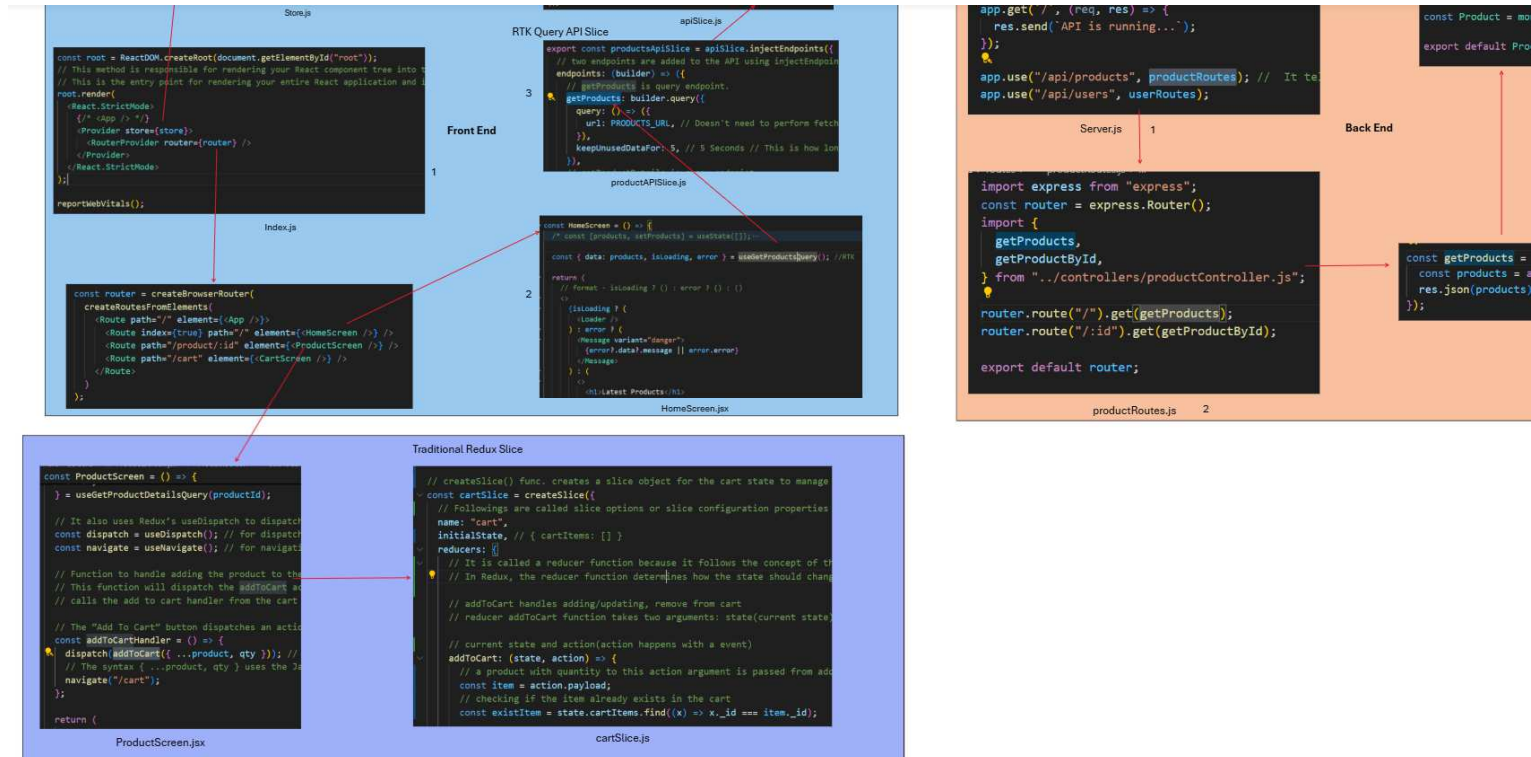
export default router;

const getProducts = asyncHandler(async (req, res) => {
 const products = await Product.find({});
 res.json(products);
});

const Product = mongoose.model("Product", productSchema);
export default Product;
```

# Product Screen Loading

Sunday, June 15, 2025 1:59 PM



# Express: req, res

Saturday, June 7, 2025

4:38 PM

The `req` and `res` objects are **automatically provided by Express** when a request is made to your route.

## How it works:

- When you define a route like:

```
router.post("/login", authUser);
```

- And your controller is:

```
const authUser = asyncHandler(async (req, res)
```

- **Express** calls your handler function and passes in the `req` (request) and `res` (response) objects for you.

**You do not need to import or define `req` and `res` yourself.**

They are created by Express for every incoming HTTP request and passed to your route handler automatically.



## --prefix

Friday, August 1, 2025

8:40 PM

```
"build": "npm install && npm install --prefix frontend && npm run build --prefix frontend"
```

--prefix <folder> tells npm to execute the command in the specified folder, not the current one

# React Bootstrap

Thursday, July 17, 2025 7:13 PM

- Form Container

○

```
<Form>
 <Form.Group>
 <Form.Label></Form.Label>
 <Form.Control></Form.Control>
 </Form.Group>
</Form>
```

# Build & Deploy

Friday, August 1, 2025

11:11 PM

# Frontend Build

Friday, August 1, 2025

11:12 PM

why frontend build necessary?

JS server.js:34-65

> Used 1 reference

The **frontend build** is necessary because your React app (in the `frontend` folder) is written in modern JavaScript (JSX, ES6+) and uses tools like React, which browsers cannot run directly. The build process:

- 1. **Compiles and bundles** your React code into plain JavaScript, HTML, and CSS files that browsers understand.
- 2. **Optimizes** the code for performance (minifies, removes unused code, etc.).
- 3. **Outputs** the result into a `build` folder (usually `frontend/build`).

When you deploy your app, the backend Express server serves these static files to users.

Without the build, users would not be able to load your React app from the server.

## Summary:

The frontend build turns your React code into static files that browsers can run, allowing your backend to serve your app to users in production.

what frontend build gonna include

JS server.js:39-68

> Used 1 reference

The **frontend build** (the `frontend/build` folder) will include:

1. **index.html**

The main HTML file that loads your React app in the browser.

2. **Minified JavaScript files**

Bundled and optimized JavaScript code for your entire React app (including all components, libraries, and dependencies).

3. **CSS files**

All your styles, combined and optimized.

4. **Static assets**

Images, fonts, icons, and any other files in your `public` folder that are needed by your app.

5. **Asset manifest**

A file (like `asset-manifest.json`) that maps original filenames to their hashed versions for cache busting.

---

**In summary:**

The frontend build contains everything needed to run your React app in the browser, optimized for fast loading and production use. Your Express server serves these files to users when they visit your site.



why my react components are not included in this build folder

JS server.js:39-68

> Used 1 reference

Your **React components are included in the build folder**, but not as individual `.js` or `.jsx` files. Instead, when you run `npm run build` in your React app, **all your components and code are bundled and minified into a few static JavaScript files** (like `main.[hash].js`) by the build tool (e.g., `react-scripts/Webpack`).

Why you don't see your components as files:

- The build process **combines all your React components, dependencies, and logic into optimized bundles**.
- These bundles are placed in the `build/static/js` directory (and referenced from `index.html`).
- **You will not see your original component files** (like `ProductScreen.jsx`) in the build folder. Instead, their code is inside the minified bundle files.

What the build folder contains:

- `index.html` (the entry point for your app)
- `static/js/main.[hash].js` (all your React code, including components, bundled together)
- `static/css/` (your styles)
- Other assets (images, fonts, etc.)

How it works:

When a user visits your site, the server sends `index.html` and the bundled JS/CSS files. The browser loads the JS bundle, which contains all your React components and logic, and renders your app.

#### Summary:

Your React components **are** in the build folder, but as part of the bundled and minified JavaScript files—not as separate files. This is normal and expected for React production builds.

> Used 1 reference

Building a **production build** is necessary because:

1. **Browsers can't run your source code directly:**

Your React app uses JSX, ES6+, and module imports, which browsers do not understand. The build process transpiles and bundles your code into plain JavaScript, HTML, and CSS that browsers can run.

2. **Performance optimization:**

The build process minifies your code, removes unused code (tree-shaking), and optimizes assets. This results in smaller file sizes and faster load times for users.

3. **Single entry point:**

The build process creates a single `index.html` and a few static JS/CSS files, making it easy for your backend (like Express) to serve your entire app.

4. **Cache busting:**

Filenames include content hashes, so browsers always load the latest version after a deployment.

5. **Security:**

The build hides your source code structure and development tools, exposing only what's needed to run the app.

**Summary:**

A production build ensures your React app is fast, compatible with all browsers, secure, and easy to deploy and serve from your backend server.



# Backend

Friday, August 1, 2025

11:25 PM

ok we have been talking about the frontend folder, so this folder completely gonna serve for client user after the build, and what about backend folder? where it gonna reside.

JS server.js:39-68

> Used 1 reference

Great question! Here's how it works in a typical MERN stack deployment:

## Frontend ( frontend folder)

- **Purpose:** Contains your React app (client-side code).
- **After build:** The `frontend/build` folder contains static files (HTML, JS, CSS, images) that are served to the user's browser.
- **Served by:** Your Express backend (see your `JS server.js`), or by a static file host (like Vercel, Netlify, or S3) if you separate frontend and backend.

## Backend ( backend folder)

- **Purpose:** Contains your Express server, API routes, database logic, authentication, etc.
- **Where it runs:**
  - On your server (e.g., a VPS, cloud VM, or a platform like Heroku, Render, Railway, etc.).
  - The backend is a Node.js process that listens for HTTP requests (API calls, file uploads, etc.).

- **How it interacts with frontend:**

- Serves the built frontend files (from `frontend/build`) to users.
- Handles API requests from the frontend (e.g., `/api/products`, `/api/users`).
- Manages database operations, authentication, etc.

---

## Deployment Structure

- **On your server:**

- The entire project (both `backend` and `frontend` folders) is uploaded.
- You run the backend server (e.g., `node backend/server.js` or with a process manager like PM2).
- The backend serves both the API and the static frontend build.

- **Alternative:**

- You can deploy the backend and frontend separately (e.g., backend on Heroku, frontend on Netlify), but in your current setup, the backend serves the frontend build.

---

## Summary:

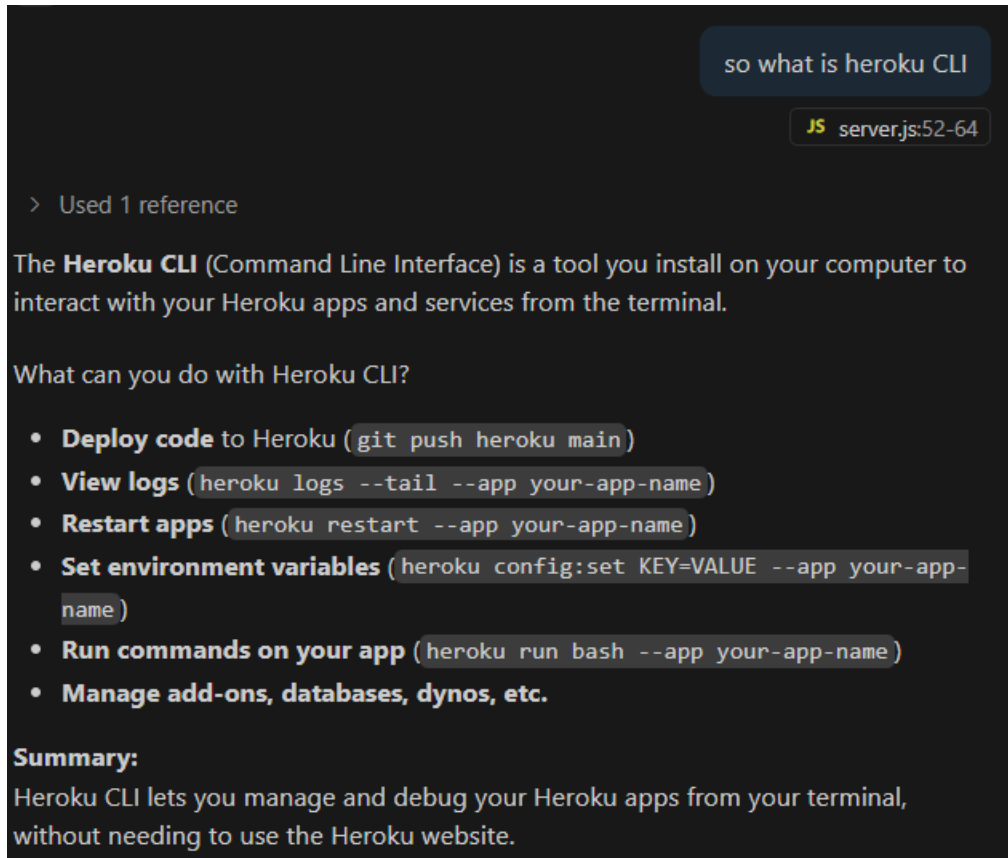
- The `frontend/build` folder is what the client (browser) downloads and runs.
- The `backend` folder is your server-side code, running on your server, handling APIs and serving the frontend build to users.

Let me know if you want deployment examples or a visual diagram!

# Heroku CLI

Saturday, August 2, 2025 4:52 PM

- What is heroku CLI



The screenshot shows a VS Code editor window with a file named `server.js:52-64`. The document content is as follows:

> Used 1 reference

The **Heroku CLI** (Command Line Interface) is a tool you install on your computer to interact with your Heroku apps and services from the terminal.

What can you do with Heroku CLI?

- **Deploy code** to Heroku (`git push heroku main`)
- **View logs** (`heroku logs --tail --app your-app-name`)
- **Restart apps** (`heroku restart --app your-app-name`)
- **Set environment variables** (`heroku config:set KEY=VALUE --app your-app-name`)
- **Run commands on your app** (`heroku run bash --app your-app-name`)
- **Manage add-ons, databases, dynos, etc.**

**Summary:**  
Heroku CLI lets you manage and debug your Heroku apps from your terminal, without needing to use the Heroku website.

- What happens running: `heroku logs --tail --app techmartx`

when i run heroku logs --tail --app techmartx on vscode, does the code push to github and the deploy on heroku online and my website get update>

JS server.js:52-64

> Used 1 reference

No, running

```
heroku logs --tail --app techmartx
```

**does not push code to GitHub, deploy to Heroku, or update your website.**

This command **only shows the live logs** from your Heroku app.  
It is for debugging and monitoring, not for deployment.

---

**To update your website on Heroku:**

1. **Save your code changes.**
2. **Commit your changes to Git (locally):**

```
git add .
git commit -m "Your message"
```

3. **Push your code to Heroku:**

```
git push heroku main
```

- View Logs Command
  - heroku logs --tail --app techmartx
- Restart Heroku App
  - heroku restart --app techmartx

# Errors

Saturday, August 2, 2025 4:57 PM

## Faced Errors While Deploying

- Helmet was only working with up to react v18. Current project was using v19. So had to ' `npm install --legacy-peer-deps`

```
"build": "npm install --legacy-peer-deps && npm install --prefix frontend --legacy-peer-deps && npm run build --prefix frontend"
```

- In server.js
  - a. dotenv package was importing and using `dotenv.config()`; and dotenv has installed as a *dev dependency* in package.json file but it only needed to be on development, not in production,
  - b. But even in production server.js file has having those statements import dotenv and `dotenv.config`
  - c. So had to install dotenv in dependency in package.json and remove from dev dependency. Then it worked!.
- Migrating from Express v4 to v5
  - In this error even github copilot couldn't get it. It was providing solutions but didn't really get close to mentioning the express v5 migrating. Only when googling and reading through the stackoverflow mentioned to see the express v5 migrating docs. Note That!

```
app.get("/*splat", (req, res) => {
 res.sendFile(path.resolve(__dirname, "frontend", "build", "index.html"))
});
} else {
 app.get("/", (req, res) => {
 res.send(`API is running...`);
 });
}
```

Official Docs: [Migrating to Express 5](#)

## Changed

### Path route matching syntax

Path route matching syntax is when a string is supplied as the first parameter to the `app.all()`, `app.use()`, `app.METHOD()`, `router.all()`, `router.METHOD()`, and `router.use()` APIs. The following changes have been made to how the path string is matched to an incoming request:

- The wildcard `*` must have a name, matching the behavior of parameters `:`, use `/*splat` instead of `/*`

```
// v4
app.get('/*', async (req, res) => {
 res.send('ok')
})

// v5
app.get('/*splat', async (req, res) => {
 res.send('ok')
})
```

### Note

`*splat` matches any path without the root path. If you need to match the root path as well `/`, you can use `/{*splat}`, wrapping the wildcard in braces.

```
// v5
app.get('/{*splat}', async (req, res) => {
 res.send('ok')
})
```

# Domain

Sunday, August 3, 2025 11:09 AM

just enlighten me about domain and domain names and how to get one and connect with heroku

ProductCarousel.jsx:1-17

> Used 1 reference

What is a domain name?

A **domain name** is a human-readable address for your website (like `techmartx.com`).

It points to your web server so users can access your site easily.

How to get a domain name

**1. Choose a domain registrar:**

Examples: [Namecheap](#), [GoDaddy](#), [Google Domains](#), [Cloudflare](#)

**2. Search for your desired domain name** and buy it.

How to connect your domain to Heroku

**1. Add your domain to Heroku:**

```
heroku domains:add yourdomain.com --app your-her
```

Or use the Heroku dashboard (Settings > Domains).

**2. Get the DNS target from Heroku:**

Heroku will give you a DNS target like `your-app-name.herokuapp.com`.

**3. Go to your domain registrar's DNS settings:**

- Add a **CNAME record** for `www` pointing to Heroku's DNS target.
- For root domains (`yourdomain.com`), use an **ALIAS** or **ANAME** record if supported, or redirect root to `www`.

**4. Wait for DNS to propagate** (can take up to 24 hours).

**Summary:**

- Buy a domain from a registrar.
- Add the domain to Heroku.
- Point your domain's DNS to Heroku's DNS target.
- Your site will be live on your custom domain!

## What is DNS



what is DNS?

 ProductCarousel.jsx:1-17

> Used 1 reference

**DNS** stands for **Domain Name System**.

It is like the internet's phonebook: it translates human-friendly domain names (like `techmartx.com`) into IP addresses that computers use to find each other.

**Example:**

When you type `google.com` in your browser, DNS finds the IP address for Google's servers so your browser can connect to the right place.

**Summary:**

DNS connects domain names to server IP addresses, making websites easy to access for humans.

Domain for free

> Used 1 reference

Yes, you can get a **free domain**, but usually only with certain limitations:

#### Common Free Domain Options

- **Freenom** ([freenom.com](https://freenom.com)):  
Offers free domains with extensions like `.tk`, `.ml`, `.ga`, `.cf`, `.gq`.
- **GitHub Student Pack**:  
If you're a student, you can get a free `.me` domain for one year via Namecheap.
- **Subdomains**:  
Services like [Netlify](#), [Vercel](#), and [Heroku](#) give you a free subdomain (e.g., `yourapp.herokuapp.com`), but not a custom top-level domain.

---

#### Limitations:

- Free domains may look less professional.
- They may have restrictions or ads.
- They can be revoked at any time.

#### Summary:

You can get a free domain from Freenom or as a student, but for a professional site, it's best to buy a domain from a registrar.

# Small Bug fixes

Sunday, August 3, 2025 11:17 AM

## 4. Fix custom errors and checking For valid ObjectId

In the `backend/middleware` folder, create a new file called `checkObjectId.js` and add the following code:

```
import { isValidObjectId } from 'mongoose';

function checkObjectId(req, res, next) {
 if (!isValidObjectId(req.params.id)) {
 res.status(404);
 throw new Error(`Invalid ObjectId of: ${req.params.id}`);
 }
 next();
}

export default checkObjectId;
```

This will check if the `req.params.id` is a valid Mongoose ObjectId and throw an error if it is not.

Now in the `backend/middleware/errorMiddleware.js` file, remove the following code as we will not be using it:

```
// If Mongoose not found error, set to 404 and change message
if (err.name === 'CastError' && err.kind === 'ObjectId') {
 statusCode = 404;
 message = 'Resource not found';
}
```

Now go into the product routes file - `backend/routes/productRoutes.js` and bring the `checkObjectId` function in:

```
import checkObjectId from '../middleware/checkObjectId.js';
```

We are going to use this in the routes that have an `id` parameter.

```
router.route('/:id/reviews').post(protect, checkObjectId, createProductReview);

router
 .route('/:id')
 .get(checkObjectId, getProductById)
 .put(protect, admin, checkObjectId, updateProduct)
 .delete(protect, admin, checkObjectId, deleteProduct);
```

Now if you go to your browser and enter an invalid `id` parameter, such as `http://localhost:3000/product/123`, you should get the following error:

```
{
 "message": "Invalid ObjectId of: 123"
}
```

## 6. User inherits the last user's cart and shipping

Right now, if a user logs out and another user logs in, the new user will inherit the last user's cart and shipping address. We need to fix this by making sure that it is cleared when a user logs out.

Go to the `frontend/src/slices/authSlice.js` file and replace the following code:

```
localStorage.removeItem('userInfo');
localStorage.removeItem('expirationTime');
```

with:

```
localStorage.clear();
```

Then go to the `frontend/slices/cartSlice.js` file and add a new `resetCart` reducer:

```
// ..
clearCartItems: (state, action) => {
 state.cartItems = [];
 localStorage.setItem('cart', JSON.stringify(state));
},
resetCart: (state) => (state = initialState), // Add this line
```

export it:

```
export const {
```

# Payment Bug Fixes

Sunday, August 3, 2025 2:12 PM

- Is paid
  - Nerd can set it to true and get items for free,
  - So we are going to ask for PayPal if the payment is done.
- Total Price
  - Currently calculating the total price on client side, but a nerd can set it tax, shipping etc. price to 0 and take items for free
  - So going to calculate Total on server side

# Concepts

Thursday, May 15, 2025

5:56 PM

# What is API

Saturday, May 24, 2025 11:04 AM

## API stands for Application Programming Interface.

### In simple terms:

**An API is a set of rules and tools that allows two software applications to communicate with each other.**

- Real-life analogy:

Imagine a restaurant:

You (the customer) are the client.

The kitchen is the server.

The waiter is the API — they take your order, bring it to the kitchen, and deliver the food back to you.

- In software:

A client app (like a mobile app or website) sends a request to the API.

The API passes this request to the server (back-end system).

The server processes the request and sends the response back through the API.

Example:

A weather app uses an API to get current weather data:

http

Copy

Edit

GET <https://api.weather.com/current?city=London>

The API returns:

json

Copy

Edit

```
{
 "city": "London",
 "temperature": "15°C",
 "condition": "Cloudy"
}
```

APIs are everywhere — they power login systems, maps, payments, social media, and more.

Let me know if you want a deeper explanation or an example in a specific programming language!

## APi in web developing

**In web development, an API (Application Programming Interface) is a bridge between the front end (client-side) and the back end (server-side). It allows web apps to send and receive data.**

🔗 What does an API do in web development?

Lets the frontend (HTML/CSS/JavaScript) talk to the backend (Node.js, Python, PHP, etc.)

Sends requests (like "get user info") and receives responses (like "here is the user data")

Often uses HTTP(S) and follows REST or GraphQL standards

# Authenticate Users

Wednesday, June 4, 2025 11:28 PM

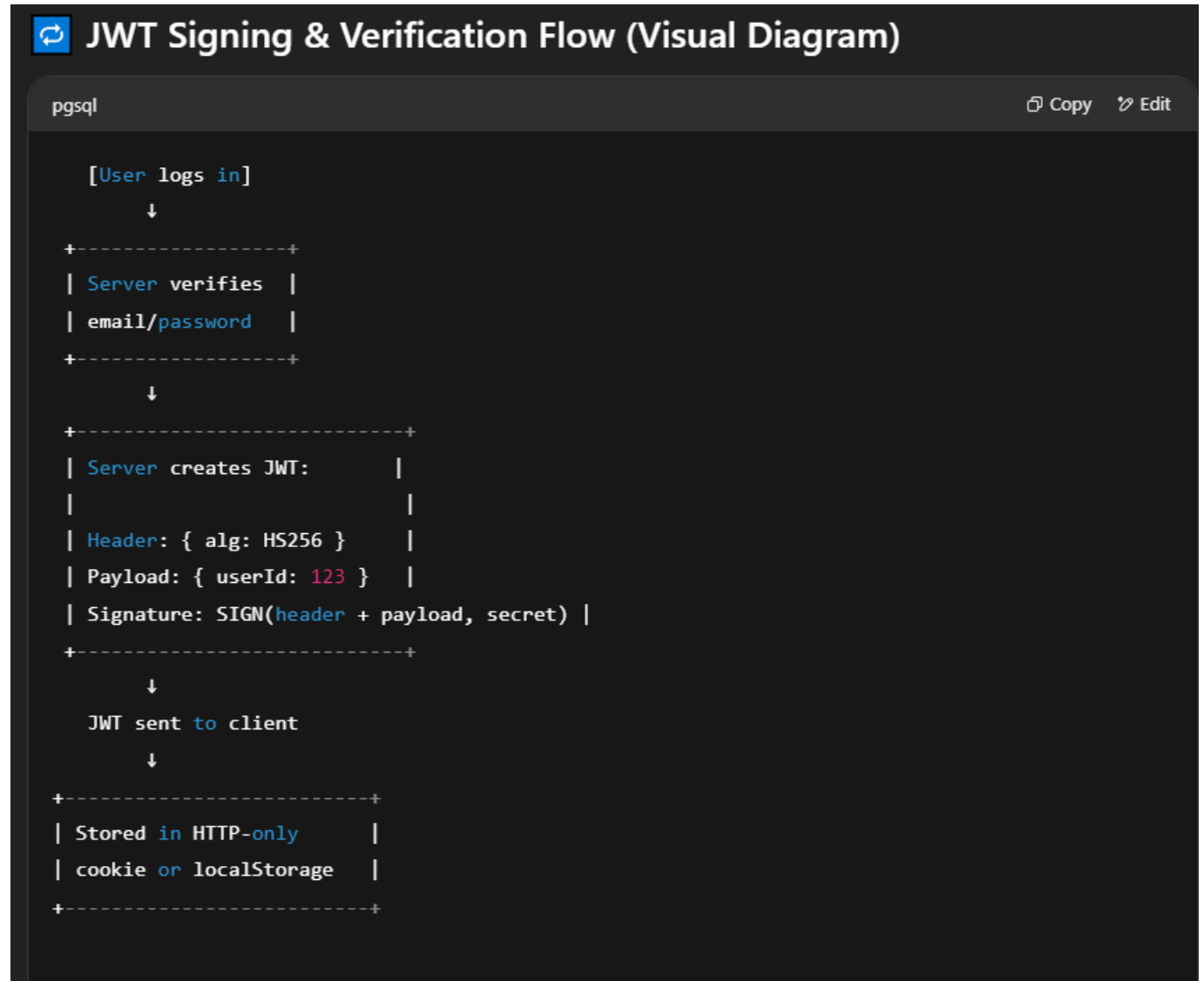
- In web development there are many ways to authenticate a user/ verifying user's identity
  - Cookies
  - Sessions
  - Json web tokens (JWT)
  - Or combination
  - There are other services like Auth
  - 3rd party services like Auth0
- Json web tokens (JWT)
  - Secure way to share information between two parties, web server and client
  - Consist of three parts
    - Header
    - Payload
      - Users id/ role
    - Signature
      - Verify information if it has tampered (interfered) in any way.
  - Traditionally Json web tokens often get stored in browser on the client. (local storage) and send it to any protective routes where needs to be logged in/ authenticate.
  - That's not the best way to do things
    - It can be insecure, because its stored in client, can be opened yourself up to cross site scripting attacks, etc.
  - Go to method.
    - Token get store in HTTP only cookie on the server.

# JWT - JSON Web Tokens

Saturday, June 7, 2025

12:48 PM

- ChatGPT: [JSON Web Tokens](#)



>>> Later, the **user** sends request **with** JWT

```
↓
+-----+
| Server receives request |
| with JWT |
+-----+
↓
+-----+
| Server verifies signature: |
| SIGN(header + payload, secret) == token.signature ? |
+-----+
↓
✅ Yes → User is trusted
❌ No → Reject the request
```

## 🧠 What Problem Does JWT Solve?

In web apps, we often need to **remember who the user is** after they log in.

Imagine this:

User logs into your website → Server says: "Yes, you're John, welcome!" → Now, **how do we remember John on every new page or request?**

That's where **authentication** comes in. And one common way to do this today is with a **JWT**.

## 🔄 Traditional Way vs JWT

### A Traditional (Session-based Auth):

- Server stores a "session" in memory or a database.
  - Server gives the client a session ID (like a ticket).
  - Client sends this ID back on every request.
  - Server checks the session ID in its storage.
- ✅ Easy  
❌ Doesn't scale well (server has to remember every session)

## 🧠 Your Core Question:

If the server sends a JWT to the client and it includes a signature, then:

## Your Core Question:

If the server sends a JWT to the client and it includes a **signature**, then:

**Why can't an outsider (or hacker) get the secret key from that signature?**

## Short Answer:

Because the signature is the result of a one-way function — you can verify it, but you can't reverse-engineer it to get the secret key.

## Why can't the secret be extracted?

Hashing functions like HMAC-SHA256 are **designed to be irreversible**.

Even if someone sees the final hash (signature), they cannot "work backward" to find the secret.

It's like making **smoothie juice**:

- You put in fruits (data + secret)
- You blend it (HMAC)
- You get juice (signature)

But you **can't reverse the juice** to get the exact fruit ingredients back.

## Traditional (Session-Based) Authentication Flow

## Traditional (Session-Based) Authentication Flow

Here's how it typically works:

### 1. User Logs In

- User sends email and password to the server.
- Server **checks the credentials** in its database.

### 2. Server Creates a Session

- Server **creates a record** in its memory/database:

```
json Copy Edit
{
 "sessionId": "abc123",
 "userId": "42",
 "createdAt": "2025-06-07T08:00:00Z"
}
```

### 3. Server Sends a Cookie

- Server sends back a **session ID cookie**:

```
http Copy Edit
Set-Cookie: sessionId=abc123; HttpOnly; Secure;
```

### 4. Next Request → Client Sends Cookie

- The client automatically sends the session ID cookie with every request.

### 5. Server Verifies Session

- On every request, the server:
  - Looks up `sessionId` in its session store.
  - If found → retrieves the user info and trusts them.
  - If not found → denies access (session expired or tampered).

## Why Is This Called **Stateful**?

Because:

- The **server is maintaining state** (user sessions) **between requests**.
- The server **has to remember who you are** by storing a session object.
- If the server crashes or is restarted — **sessions may be lost** unless stored in a shared database (like Redis).

## Summary Table: Why It's Stateful

### What Happens

Server stores session ID

Server looks it up every time

Crashes can lose session data

Can't scale easily to many servers

### Why It's Considered State

Server holds state data

Relies on internal memory/db

State needs to be persisted

State isn't shared by default



## Real-Life Analogy

**Stateful = Like a hotel with a guest book**

1. You check in → they write your name in their book.
2. You come back later → they look up your name.
3. If the book is lost (server down), they don't remember you.

## Pros and Cons of Stateful Auth

### Pros

Simple to implement

Can easily invalidate sessions

Common for web apps

### Cons

Harder to scale

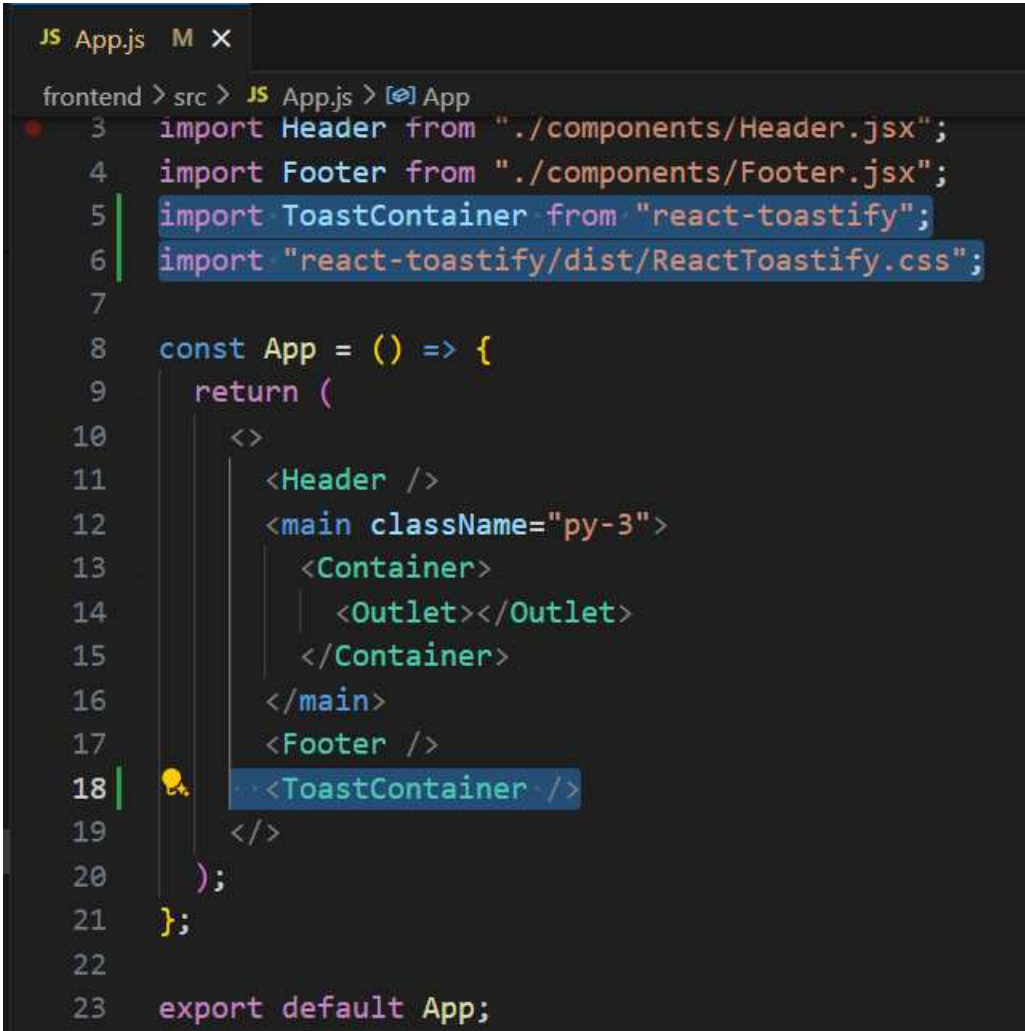
Server must track all users

Session storage required

# Packages Installed

Thursday, June 19, 2025 11:21 AM

- Toastify - Alert



```
JS App.js M X
frontend > src > JS App.js > [App] App
3 import Header from "../components/Header.jsx";
4 import Footer from "../components/Footer.jsx";
5 import ToastContainer from "react-toastify";
6 import "react-toastify/dist/ReactToastify.css";
7
8 const App = () => {
9 return (
10 <>
11 <Header />
12 <main className="py-3">
13 <Container>
14 <Outlet></Outlet>
15 </Container>
16 </main>
17 <Footer />
18 <ToastContainer />
19 </>
20);
21 };
22
23 export default App;
```

- jsonwebtoken
- cookie-parser

- React-bootstrap
- Bootstrap
- React-icons

- **npm I -D nodemon**

- D means it will only install in development environment, not will end up in production.

- nodemon - watch the file and restart the server on change
- concurrently - make it able to run two servers at same time

```
"scripts": {
 "start": "node backend/server.js",
 "server": "nodemon backend/server.js",
 "client": "npm start --prefix frontend",
 "dev": "concurrently \"npm run server\" \"npm run client\""
},
```

-

# Git Initialize

Saturday, March 22, 2025 2:07 PM

- The files and folders that are not needed to be included to a git repository go .gitignore file.
- Pull the .gitignore file out of the frontend folder to root folder. So we can ignore file in other directories like backend folder and etc.
- And make changes in .gitignore file

```
3 # dependencies
4 node_modules
5 /.pnp
6 .pnp.js
7
8 # testing
9 /coverage
10
11 # production
12 /build
13
14 # misc
15 .DS_Store
16 .env
17 .env.local
18 .env.development.local
19 .env.test.local
20 .env.production.local
```

- In the bash shell
  - `ls -a` : you can see hidden folders there
  - Remove .git repository folder using `'rm -rf .git'`
- Now initialize a new git repository in the root folder (instead frontend folder)
  - `'git init'`
- Create a GitHub Repository
- And setup the connection
  - Instead of add README.md perform add . to add all changes.

```
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@your_gitrepo.git
```

```
git push -u origin main
```

# npm init

Thursday, June 19, 2025

12:05 PM

- npm init

 **What is `npm init`?**

`npm init` is a command that:

- Initializes a new Node.js project by creating a file called `package.json`.

---

 **What is `package.json`?**

It's a file that:

- Describes your project (name, version, author, etc.)
- Tracks dependencies (e.g., React, Express, etc.)
- Helps `npm` (Node Package Manager) manage and install packages for your app

- **When do you use npm init?**

You use it **at the beginning of a new project**, when you're setting up your project folder.

- Why change to - Type : module in package JSON

Excellent question! The `"type"` field in `package.json` tells Node.js how to interpret your JavaScript files — either as:

- **ES Modules (ESM):** `"type": "module"`
- **CommonJS (CJS):** `"type": "commonjs"` (default if you don't specify anything)

Let's compare them clearly:

## 1. `"type": "commonjs"` (the traditional/default format)

- Default in Node.js
- Uses `require()` and `module.exports`
- Common in older Node.js apps and most npm packages

### Example:

```
js Copy Edit
// commonjs.js
const fs = require('fs'); // import
module.exports = {}; // export
```

## 2. `"type": "module"` (the modern ES module format)

- Uses `import` and `export` (like in modern browser JS or React)
- Required if you want to use `import` / `export` syntax in Node.js without transpilers (e.g., Babel)

### Example:

```
js Copy Edit
// esm.js
import fs from 'fs'; // import
export default {}; // export
```

If you use `"type": "module"`, Node expects `.js` files to use ES Module syntax.

Ex:

```
{ } package.json U JS server.js U X
backend > JS server.js > ...
1 import express from "express"; // "type: module"
2 const express = require("express"); // "type: commonjs"
3
```

# VS Code

Thursday, June 19, 2025

12:02 PM

# Multiple Line Comment

Saturday, May 24, 2025 10:44 AM

- Multiple line comment - Shift + Alt + A

# Eslint

Wednesday, July 16, 2025 11:03 PM

- Installed Eslint for visible errors display in JS, react app
- Customize eslint file.